iOS Sale Discount Service Guide

The Merchant Sale Discount Service provides all sale discounts-related operations—such as retrieval, creation, updating, and deletion of sale discount data.

Below are the key utility features available:

- **Create Merchant Sale Item Discounts**: Creates one or more sale item discounts under the merchant's account.
- **Get All/Specific Sale Item Discounts**: Processes the get merchant discount request and retrieves the specified discounts for the merchant.
- **Update Existing Sale Item Discounts**: Processes the update merchant sale discount request and applies the specified changes to the merchant's sale discount.
- **Delete Merchant Sale Item Discount**: Processes the Delete Merchant Sale Discount request and removes the specified sale discount for the merchant.

Prerequisites

Before you continue, ensure you have the following prerequisites completed and ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and retrieve associated merchant accounts, which are necessary to establish a connection with the terminal.
  - iOS Merchant Authorization & Accounts Guide

ON THIS PAGE

Prerequisites

Create Merchant Sale Discount

Get All/Specific Sale Item Discounts

Update Existing Sale Item Discounts

Delete Merchant Sale Item Discount

Create Merchant Sale Discount

Creates one or more sale item discounts under the merchant's account.

Step 1: Create a MerchantDiscountService Object

```
let discountService = MerchantDiscountService()
```

Step 2: Create a Merchant Discount

Construct a MerchantDiscountRequest by providing the required input parameters, including a discountList which is a list of Discount objects representing the discounts to be created for the merchant.

```
let discount = Discount(
discountId: "DISC1001",
discountName: "Summer Sale",
isPercent: true,
discountValue: 15.0,
isEnabled: true
)
let discountRequest = MerchantDiscountRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
userId: "",
discountList: [discount1,discount2]
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

Pass the request and bearer token to the insertMerchantDiscount() method.

```
discountService.insertMerchantDiscount(request:
request, bearerToken: bearerToken) { response,
error in
if let result = response {

} else if let error = error {

}
}
```

Step 3: Response Handling

On success, insertMerchantDiscount() method returns a MerchantDiscountRespons object which contains the following:

- **If success = true**, response contains list of Discount objects that represent the discounts created for the merchant.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Get All/Specific Sale Item Discounts

Takes the get merchant discount request and then retrieves the requested discounts for the merchant. This can be used to fetch either particular discount details or the entire discounts for the requested user.

Step 1: Create a MerchantDiscountService Object

```
let discountService = MerchantDiscountService()
```

Step 2: Get Merchant Sale Discount

The GetMerchantDiscountRequest object is used to define the data required to fetch merchant sale discounts for a merchant account, where discountID specifies the particular discount to retrieve; if discountID is null, all discounts for the merchant user are fetched.

```
let request = GetMerchantDiscountRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
userId: "",
discountID: "123"
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

```
discountService.getMerchantDiscount(request:
request, bearerToken: bearerToken) { response,
error in
if let result = response {
} else if let error = error {

}
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

On success, getMerchantDiscount() method returns a GetMerchantDiscountResponse object containing the following:

- **If success = true**, response contains list of Discount objects that represent the discounts created for the merchant.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Update Existing Sale Item Discounts

Processes the update merchant sale discount request and applies the specified changes to the merchant's sale discount.

Step 1: Create a MerchantDiscountService Object

```
let discountService = MerchantDiscountService()
```

Step 2: Update Merchant Sale Discount

The MerchantDiscountRequest object is used to define the data required to update sale discounts for a merchant account, where discountList contains a list of Discount objects representing the discounts to be updated.

```
let discount = Discount(
discountId: "DISC1001",
discountName: "Summer Sale",
isPercent: true,
```

```
79ab84fd3011",
userId: "",
discountList: [discount1,discount2]
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

Pass the request and bearer token to the updateMerchantDiscount() method.

```
discountService.updateMerchantDiscount(request:
updateRequest, bearerToken: bearerToken) {
response, error in
if let result = response {

} else if let error = error {

}
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

On success, updateMerchantDiscount() method returns a MerchantDiscountRespons object which contains the following:

- **If success = true**, response having list of Discount objects that represent the discounts created or updated for the merchant.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Delete Merchant Sale Item Discount

Takes the delete merchant sale discount request and then deletes the requested merchant sale discount.

Step 1: Create a MerchantDiscountService Object

Step 2: Delete a Merchant Sale Discount

The GetMerchantDiscountRequest object is used to define the data required to delete sale discounts for a merchant account, where discountID specifies the specific discount to delete; if null, all discounts for the merchant user are deleted.

```
let deleteRequest = GetMerchantDiscountRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
userId: "",
discountID: "123"
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

Pass the request and bearer token to the deleteMerchantDiscount() method.

```
discountService.deleteMerchantDiscount(request:
deleteRequest, bearerToken: bearerToken) {
response, error in
if let result = response {

} else if let error = error {

}
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

Step 3: Response Handling

On success, deleteMerchantDiscount() method returns a DeleteMerchantDiscountResponse object containing the following:

- **If success = true**, response contains list of Discount objects that represent the discounts created for the merchant.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

deluxe. connect

Payments API

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave,
Minneapolis, MN 55402.

© 2026 Deluxe Corporation